

Lab 3: Frontend, Ingress & Persistent Storage

From a Blank Folder to a Complete 3-Tier Stack Behind One Entry Point

DONE BY: DHARSHINI K

What You Will Build

By the end of this lab you will have created an Angular application from scratch, containerized it, and deployed it to Kubernetes alongside your existing PostgreSQL and .NET Core API from Lab 2. A single Ingress will route browser traffic to either the frontend or the backend based on URL path. PostgreSQL will be backed by a PersistentVolume so its data survives Pod restarts — closing the gap deliberately left open in Lab 1.

Learning Objectives

- Scaffold and build an Angular application for production
- Containerize a static frontend using a multi-stage Docker build with nginx
- Install and understand the role of an Ingress Controller
- Configure path-based routing through a single Ingress resource
- Understand PersistentVolumes, PersistentVolumeClaims, and why Pods alone cannot retain data

Prerequisites

- Node.js and npm installed (node --version, npm --version)
- Angular CLI installed globally: npm install -g @angular/cli
- Lab 2 completed: postgres and api Pods running and reachable via /db-check
- Docker Desktop with Kubernetes enabled

Part A — Create the Angular Application

Step A1 — Scaffold a new Angular project

Run this from a clean working directory, outside your MyApi folder.

```
cd C:\k8s-training
ng new my-angular-app
```

When prompted, choose the defaults: no SSR/SSG (Server-Side Rendering) unless you specifically want it — this lab assumes a plain client-side rendered app, which is the simpler and more common starting point. Choose CSS for styling unless you have a preference.

This creates a `my-angular-app` folder with a working starter application.

```

dharshinik@DHC47TYX72-dharshinik frontend_backend_db % ng new my-angular-app
✓ Which stylesheet system would you like to use? CSS [
https://developer.mozilla.org/docs/Web/CSS ]
✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation
(SSG/Prerendering)? No
✓ Which AI tools do you want to configure with Angular best practices?
https://angular.dev/ai/develop-with-ai None
CREATE my-angular-app/.prettierrc (161 bytes)
CREATE my-angular-app/README.md (1466 bytes)
CREATE my-angular-app/.editorconfig (314 bytes)
CREATE my-angular-app/.gitignore (622 bytes)
CREATE my-angular-app/angular.json (1927 bytes)
CREATE my-angular-app/package.json (791 bytes)
CREATE my-angular-app/tsconfig.json (957 bytes)
CREATE my-angular-app/tsconfig.app.json (429 bytes)

```

Step A2 — Add a simple call to your backend API

Open `my-angular-app/src/app/app.ts` (or `app.component.ts`, depending on your Angular CLI version) and replace its contents with the following. This adds a button that calls your API's `/db-check` endpoint and displays the result, proving the full chain works once deployed.

`src/app/app.ts`

```

import { Component, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-root',
  standalone: true,
  template: `
    <div style="font-family: sans-serif; padding: 2rem;">
      <h1>Kubernetes 3-Tier Demo</h1>
      <button (click)="checkDb()">Check Database Connection</button>
      <pre>{{ result() }}</pre>
    </div>
  `,
})
export class App {
  result = signal('Click the button to test the API -> Postgres connection');

  constructor(private http: HttpClient) {}

  checkDb() {
    this.http.get('/api/db-check').subscribe({

```

```

      next: (res) => this.result.set(JSON.stringify(res, null, 2)),
      error: (err) => this.result.set('Error: ' + JSON.stringify(err.message)),
    });
  }
}

```

Step A3 — Enable HttpClient

Open `src/app/app.config.ts` and ensure `provideHttpClient()` is included, so the `HttpClient` used above actually works.

`src/app/app.config.ts`

```

import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()],
};

```

Why the URL is `/api/db-check`, not the full backend address

The Angular app deliberately calls a relative path, not a hostname or port. Once deployed, the Ingress we configure in Part C will intercept any request starting with `/api` and forward it to the backend Service. The frontend code never needs to know the backend's actual location.

Step A4 — Build for production

```

cd my-angular-app
ng build

```

Once complete, check the exact output path — this varies by Angular version:

```
ls dist/my-angular-app
```

Important

Angular 17 and later nest build output under a `/browser` subfolder (`dist/my-angular-app/browser`) to support server-side rendering setups. Older versions place files directly under `dist/my-angular-app`. Confirm which structure you have now -- you will need the exact path in Step B1.

```

dharshinik@DHC47TYX72-dharshinik my-angular-app % ls dist/my-angular-app
3rdpartylicenses.txt  browser  prerendered-routes.json

```

Part B — Containerize the Angular App

Step B1 — Create the Dockerfile

This is a multi-stage build, the same pattern used for the .NET API: stage one compiles the app, stage two serves only the compiled static files using nginx. Adjust the COPY path in stage two to match what you found in Step A4.

Dockerfile

```
# Stage 1: Build the Angular app
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Serve with nginx
FROM nginx:alpine
# Adjust this path to match your actual dist output from Step A4
COPY --from=build /app/dist/my-angular-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Step B2 — Create nginx.conf

This single setting is the most commonly missed step when containerizing any single-page application. Without it, refreshing the browser on any route other than the homepage returns a 404, because nginx looks for a literal file at that path and finds none.

nginx.conf

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Place both Dockerfile and nginx.conf in the root of my-angular-app, alongside package.json.

Step B3 — Build the image

```
docker build -t myangularapp:latest .
```

```

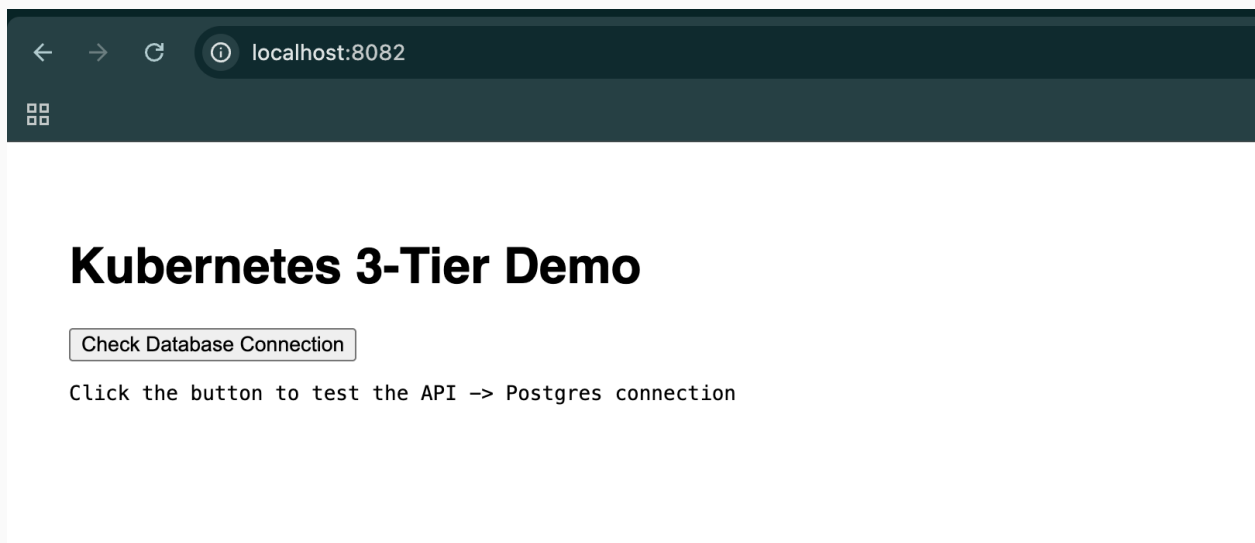
dharshinik@DHC47TYX72-dharshinik my-angular-app % docker build -t myangularapp:latest .
[+] Building 10.7s (6/14)                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 559B                                              0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine                  1.0s
=> [internal] load metadata for docker.io/library/node:20                       1.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
dharshinik@DHC47TYX72-dharshinik my-angular-app % docker images | grep myangularapp
myangularapp:latest                    fd8e3eaa0793          92MB          25.9MB

```

Step B4 — Smoke-test locally

```
docker run -p 8082:80 myangularapp:latest
```

Open <http://localhost:8082/> in a browser. You should see the page with the "Check Database Connection" button. Clicking it will fail right now with a network error -- that is expected, since `/api/db-check` has nothing to route to outside the cluster. Stop the container with Ctrl+C once confirmed the page loads.



Step B5 — Make the image visible to your cluster (kind only)

Skip this step if you chose Kubeadm

This step only applies if Docker Desktop's Kubernetes was set up using the kind engine. If you chose Kubeadm instead, your cluster shares the host's Docker image cache directly, and this step is unnecessary -- continue straight to Part C.

A kind cluster runs its node as a separate container with its own isolated container runtime and image storage, entirely separate from your host machine's Docker image cache. Building an image with docker build on your host does not automatically make it visible inside the kind node. Skipping this step is the most common cause of an ImagePullBackOff or ErrImagePull error in Part C.

First, find the exact name of your node container:

```
docker ps
```

Look for a container whose name ends in -control-plane (for example, desktop-control-plane). Then load the image directly into that node's containerd runtime:

```
docker save myangularapp:latest | docker exec -i desktop-control-plane ctr -n=k8s.io
images import -
```

Replace desktop-control-plane with your actual node container name if different. Repeat this step any time you rebuild the image -- the kind node never picks up a rebuild automatically.

```

dharshinik@DHC47TYX72-dharshinik my-angular-app % docker save myangularapp:latest | docker exec -i desktop-control-plane ctr -n=
k8s.io images import -
docker.io/library/myangularapp:latest          saved
application/vnd.oci.image.index.v1+json sha256:fd8e3eaa0793b465fe3228d56dee598f227da6a169096200635721bbe5b6c757
Importing      elapsed: 0.9 s total:    0.0 B (0.0 B/s)
dharshinik@DHC47TYX72-dharshinik my-angular-app %

```

Part C — Deploy the Frontend to Kubernetes

Step C1 — Create the frontend manifest

03-frontend.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:

```

```

    app: frontend
spec:
  containers:
    - name: frontend
      image: myangularapp:latest
      imagePullPolicy: Never
      ports:
        - containerPort: 80
      resources:
        requests:
          cpu: "50m"
          memory: "64Mi"
        limits:
          cpu: "200m"
          memory: "256Mi"
      readinessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 5
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 10
        periodSeconds: 15
  ---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP

```

Note the replica count of 2, not 1. Serving static files is cheap and entirely stateless, so there is no reason to run only a single copy from the start.

Step C2 — Deploy it

```

kubectl apply -f 03-frontend.yaml
kubectl get pods --watch

```

Wait until both frontend pods show 1/1 Running, then press Ctrl+C.

```
dhmarshinik@DHC47TYX72-dharshinik docker_to_K8 % kubectl get pods --watch
```

NAME	READY	STATUS	RESTARTS	AGE
api-65c4f494df-f4v74	1/1	Running	0	3h20m
frontend-6695bbf6b8-qtvph	1/1	Running	0	19s
frontend-6695bbf6b8-s85kx	1/1	Running	0	19s
postgres-5f64456b7d-fr6b9	1/1	Running	0	3h21m

Part D — Install an Ingress Controller

Defining an Ingress resource by itself does nothing on its own -- it is only a routing rule. Something must actually read that rule and implement the routing: that something is the Ingress Controller, a real running piece of software. Docker Desktop does not install one by default, so this step installs it explicitly.

Step D1 — Install the nginx Ingress Controller

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.11.2/deploy/static/provider/cloud/deploy.yaml
```

This deploys the controller into its own namespace, separate from your application Pods.

Step D2 — Wait for the controller to be ready

```
kubectl get pods -n ingress-nginx --watch
```

Wait for a pod named ingress-nginx-controller-... to reach 1/1 Running. This can take 30-60 seconds. Press Ctrl+C once ready.

```
dharshinik@DHC47TYX72-dharshinik docker_to_K8 % kubectl get pods -n ingress-nginx --watch
NAME                                READY   STATUS              RESTARTS   AGE
ingress-nginx-admission-create-wpt25 0/1     Completed           0           43s
ingress-nginx-admission-patch-q75fh   0/1     Completed           1           43s
ingress-nginx-controller-6d6657db77-x2pvm 0/1     ContainerCreating   0           43s
ingress-nginx-controller-6d6657db77-x2pvm 0/1     Running             0           52s
ingress-nginx-controller-6d6657db77-x2pvm 1/1     Running             0           63s
```

Part E — Configure the Ingress

A subtle bug worth understanding up front

A single Ingress resource with one rewrite-target annotation covering both the frontend and backend paths looks reasonable, but breaks the frontend in a specific way: annotations apply to the entire Ingress resource, not to individual paths within it. The rewrite-target rule, intended only to strip /api, ends up rewriting every request that flows through that Ingress -- including requests for the Angular app's own JavaScript and CSS files. The browser then receives the wrong content for those files and throws a MIME-type error. The fix is to use two separate Ingress resources, so the rewrite only ever applies to API traffic.

Step E1 — Create the frontend Ingress

This resource has no rewrite annotation at all. Requests pass through to the frontend Service exactly as received.

03-frontend-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-ingress
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
```

Step E2 — Create the API Ingress

This resource keeps the rewrite-target annotation, but it is now scoped to its own Ingress resource and therefore only ever affects API traffic. It strips the /api prefix before forwarding, so the request arrives at the backend exactly as the route definitions expect (e.g. /db-check, not /api/db-check).

03-api-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /api(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: api
                port:
                  number: 8080
```

Step E3 — Apply and verify both

```
kubectl apply -f 03-frontend-ingress.yaml
kubectl apply -f 03-api-ingress.yaml
kubectl get ingress
```

Expect to see two separate Ingress entries listed, frontend-ingress and api-ingress, both showing localhost under ADDRESS.

```

dharshinik@DHC47TYX72-dharshinik docker_to_K8 % kubectl get ingress
NAME                CLASS    HOSTS    ADDRESS    PORTS    AGE
api-ingress         nginx    *        localhost  80       12s
frontend-ingress    nginx    *        localhost  80       25s
dharshinik@DHC47TYX72-dharshinik docker_to_K8 %

```

Step E4 — Test the full stack through one entry point

Use an Incognito/InPrivate window for this test, to rule out any stale cached files from earlier attempts.

1. Open `http://localhost/` in a fresh Incognito/InPrivate window -- this should load your Angular app with no console errors.
2. Click the "Check Database Connection" button.
3. Confirm it returns a successful JSON response with the Postgres version, instead of the network error seen in Step B4.

Kubernetes 3-Tier Demo

```

Check Database Connection
{
  "status": "connected",
  "database": "appdb",
  "host": "postgres",
  "postgresVersion": "PostgreSQL 16.14 (Debian 16.14-1.pgdg13+1) on aarch64-unknown-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit"
}

```

This proves the entire chain end-to-end: browser, to Ingress, to frontend or backend Service by path, to the API Pod, to the Postgres Service by DNS name, to the Postgres Pod.

Troubleshooting Reference

Symptom	Likely Cause / Fix
nginx serves a blank page or 404 for any route	The dist output path in the Dockerfile does not match your actual Angular build output. Re-check with <code>dir dist\my-angular-app</code> and adjust the COPY line.
Refreshing on a deep link returns 404	nginx.conf is missing or its <code>try_files</code> directive was not applied. Confirm the file was actually copied into the image.
frontend pod shows ImagePullBackOff or ErrImagePull	If using kind, the image was built on the host but never loaded into the kind node's separate image store. Repeat Step B5.
kubectl get ingress shows no ADDRESS	The Ingress Controller is not yet ready, or did not install correctly. Re-check <code>kubectl get pods -n ingress-nginx</code> .
Browser console shows a MIME-type error for a .js file, page renders blank	A rewrite-target annotation is being applied to the frontend's own Ingress rule, not just the API rule. Confirm the frontend Ingress (frontend-ingress) has no rewrite annotation at all -- only api-ingress should have one.
/api/db-check returns 404 through the Ingress but works via port-forward	The rewrite-target annotation or path regex on api-ingress does not match. Confirm the path is exactly <code>/api(/ \$)(.*)</code> and the annotation is <code>/ \$2</code> .
PVC stuck in Pending	No StorageClass is available to satisfy the claim. On Docker Desktop this is rare, but confirm with <code>kubectl get storageclass</code> that a default class exists.
Postgres pod fails to start after adding the volume	Check <code>kubectl logs <pod-name></code> for a permissions or lost+found error; confirm PGDATA is set to a subfolder, not the mount root.

Key takeaway

Three separate problems, three separate Kubernetes primitives: Ingress solved "how do two different apps share one entry point," the Ingress Controller solved "who actually implements that routing," and the PersistentVolumeClaim solved "how does data outlive a Pod." None of these existed in Lab 1 by accident -- each was introduced only once its absence became a felt, concrete problem.